

# Deep RL for Automated Stock Trading

CS5242 Neural Networks and Deep Learning — Group ID24

Rutwik Shrikant Dharanappagoudar   Zhu Yufan   Fei Xin

Semester 2, 2025/26

## 1 Introduction

Stock trading is a sequential decision-making problem where an agent observes market state and chooses to *buy*, *hold*, or *sell* to maximise long-term profit. Classical rule-based strategies are brittle and unable to adapt to non-stationary market dynamics.

Deep Reinforcement Learning (DRL) learns trading policies directly from data without hand-crafted rules. We implement DQN and Double DQN (DDQN) **entirely from scratch** in PyTorch and evaluate them on historical DJIA data against a buy-and-hold baseline. The goal is to deeply understand DRL by designing and analysing these algorithms from first principles.

## 2 Data

We download daily OHLCV data via `yfinance` for DJIA stocks from 2009–2024 (~4,000 days per ticker), split chronologically into 70%/15%/15% train/val/test sets.

For each ticker we compute 9 features: (1) daily return; (2–3) short/long MA deviations; (4) RSI-14; (5) MACD signal; (6) Bollinger Band width; (7) normalised volume; (8) ATR; and (9) OBV. All features are z-score normalised using **training-set statistics only** to prevent data leakage. Raw returns and prices are preserved separately for reward computation.

## 3 Environment Design

We implement a Gym-style `TradingEnv` for single-asset trading. The state  $s_t \in \mathbb{R}^{10}$  consists of 9 normalised features plus the current position  $\text{pos}_t \in \{-1, 0, +1\}$ . The agent chooses from three discrete actions: Sell (-1), Hold (0), Buy (+1).

The reward combines PnL, transaction costs, and a

drawdown penalty:

$$r_t = \text{pos}_t \cdot \tilde{r}_t - c_t \cdot \mathbf{1}[\text{pos}_t \neq \text{pos}_{t-1}] + \lambda \min\left(0, \frac{V_t - V_t^{\text{peak}}}{V_t^{\text{peak}}}\right) \quad (1)$$

where  $\tilde{r}_t$  is the raw daily return,  $\lambda = 0.5$ , and  $c_t$  is either a flat fee (10 bps) or proportional to position change magnitude. The drawdown term encourages risk-averse behaviour.

Each episode spans one full data split. The environment tracks position history, trades, and portfolio value, and computes Sharpe ratio, max drawdown, and trade frequency at episode end.

## 4 Deep Learning Methods

**DQN** approximates  $Q^*(s, a)$  with a neural network  $Q_\theta$  trained via TD loss:

$$\mathcal{L} = \mathbb{E}_{\mathcal{D}} \left[ \left( r + \gamma \max_{a'} Q_{\tilde{\theta}}(s', a') - Q_\theta(s, a) \right)^2 \right] \quad (2)$$

Key components (all from scratch): replay buffer (10K capacity, batch size 64), target network (updated every 100 steps), and linear  $\epsilon$ -decay from 1.0 to 0.05 over 200K steps.

**DDQN** decouples action selection from evaluation to reduce overestimation:

$$y_t = r + \gamma Q_{\tilde{\theta}}(s', \arg \max_{a'} Q_\theta(s', a')) \quad (3)$$

DDQN subclasses DQN, changing only the target computation.

**Architecture.** Both use a 3-layer MLP:  $10 \rightarrow 128 \rightarrow 64 \rightarrow 3$  with ReLU activations, Adam (lr= $10^{-3}$ ), and gradient clipping (max norm 1.0).

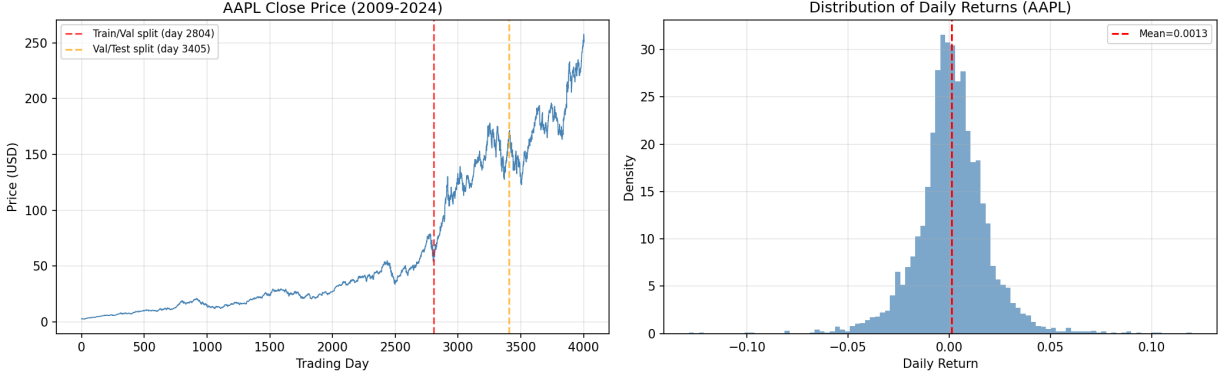


Figure 1: Left: AAPL close price (2009–2024) with train/val/test boundaries. Right: Daily return distribution showing fat tails.

## 5 Training and Results

Each agent trains for 200 episodes on the training set. Validation (greedy,  $\varepsilon=0$ ) runs every 10 episodes. Both agents show clear learning, with DDQN converging slightly faster.

Table 1: Test performance on AAPL (601 days, 2022–2024).

|             | DQN    | DDQN   | B&H    |
|-------------|--------|--------|--------|
| Cum. Return | +90.2% | +49.6% | +50.8% |
| Sharpe      | 1.23   | 0.81   | 0.82   |
| Max DD      | −25.6% | −27.1% | −28.3% |

DQN outperforms buy-and-hold on all metrics—higher return, better Sharpe, and lower drawdown—showing it learned to go flat or short during downturns and long during rallies. DDQN matches buy-and-hold with slightly lower drawdown; its conservative Q-estimates reduce both upside and downside. DQN’s mild overestimation leads to more decisive positioning, which benefits when momentum signals are genuine.

## 6 Challenges

An early bug used normalised returns ( $\text{std} \approx 1$ ) instead of raw returns for rewards, producing unrealistic values; fixed by separating `raw_return`. Exponential  $\varepsilon$ -decay exhausted exploration in one episode; linear decay over 200K steps resolved this. Short-selling can cause portfolio values below zero, so we clamp at  $10^{-8}$ .

## 7 Future Work

Extensions include policy gradient methods (PPO) for continuous position sizing, multi-asset portfolio environments, LSTM architectures for temporal dependencies, and Sharpe/CVaR-based reward formulations.

## 8 Contributions

| Member  | Contributions                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Rutwik  | DQN/DDQN implementation from scratch; hyperparameter tuning                                                                                                                                                                                                                                                                                                                                                                                                                |
| Yufan   | Data pipeline: <code>yfinance</code> OHLCV download, 9-feature engineering (RSI, MACD, Bollinger Bands, ATR, OBV, configurable MA deviations, volume normalisation), train-only z-score normalisation, multi-ticker <code>load_all()</code> support. <code>TradingEnv</code> : drawdown-penalised reward shaping (Eq. 1), flat & proportional transaction cost models, built-in Sharpe/drawdown/trade-frequency evaluation, position and portfolio tracking, episode logic |
| Fei Xin | Evaluation framework, training visualisation, report, and slides                                                                                                                                                                                                                                                                                                                                                                                                           |

## References

- [1] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, 518:529–533, 2015.

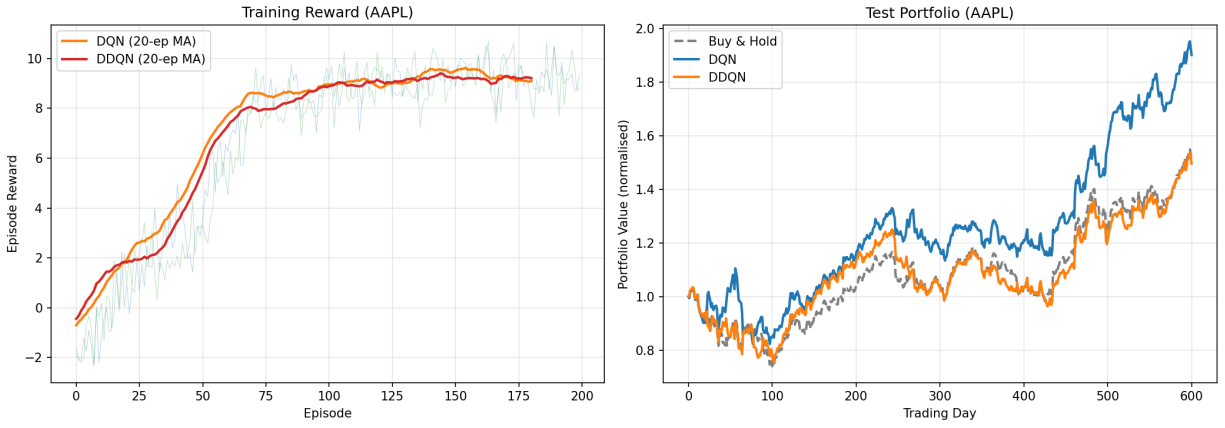


Figure 2: Left: Training reward (20-episode moving average). Right: Test portfolio value for DQN, DDQN, and buy-and-hold.

- [2] H. van Hasselt et al., “Deep reinforcement learning with double Q-learning,” *AAAI*, 2016.
- [3] H. Yang et al., “Deep reinforcement learning for automated stock trading: An ensemble strategy,” *ACM ICAIF*, 2020.